

ΜΥΕ041

Διαχείριση Σύνθετων Δεδομένων

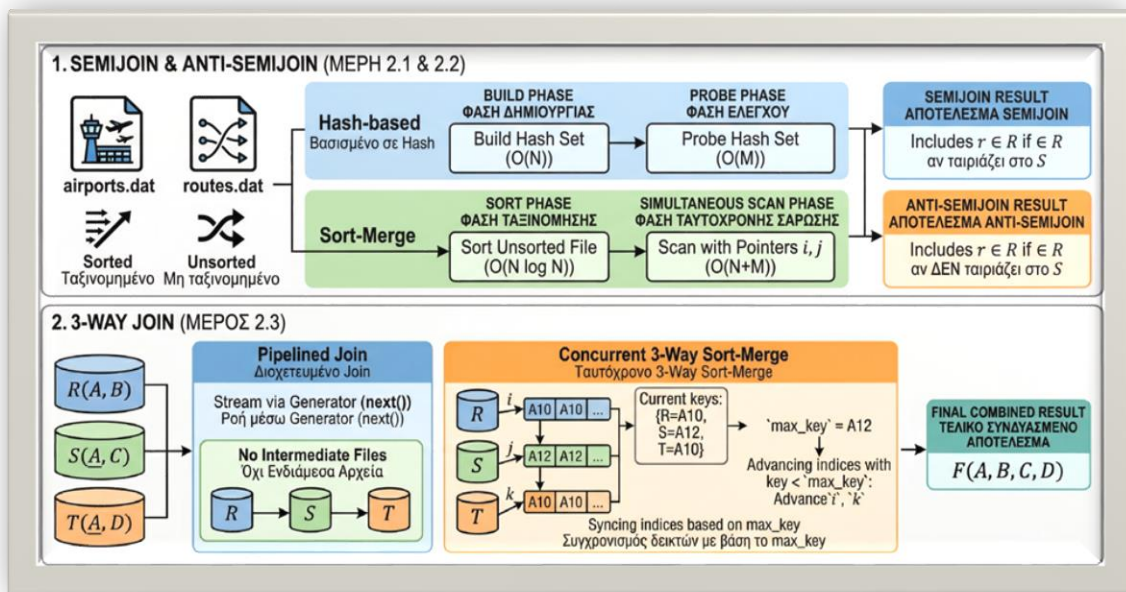
Διδάσκων: Ν. Μαμουλής

Εργασία: 1ο Σετ Ασκήσεων -
ΑΣΚΗΣΗ 2 (Αλγόριθμοι Αποτίμησης Συνενώσεων)

Ομάδα:

Παναγιώτης Παρασκευόπουλος ΑΜ: 2905

Άγγελος Μπεζαΐτης ΑΜ: 4432



Περιεχόμενα

Εισαγωγή.....	2
Προγραμματιστική Υλοποίηση και Αλγοριθμική Λογική	3
Αλγόριθμοι Hash και Sort-Merge (Μέρος 2.1)	3
Προηγηθείσα Επιλογή και Πραγματικά Δεδομένα (Μέρος 2.2)	4
Διοχέτευση (Pipelining) και 3-Way Sort-Merge (Μέρος 2.3)	4
Α) Υλοποίηση Pipelining μέσω Generators	5
Β) Ταυτόχρονος 3-Way Sort-Merge.....	5
Πειραματική Αξιολόγηση.....	5
Συμπεράσματα.....	6
Οδηγίες Εκτέλεσης (How to Run)	7

Εισαγωγή

Το πιο σημαντικό κομμάτι κάθε συστήματος βάσεων δεδομένων είναι ο τρόπος με τον οποίο εξετάζει και βελτιώνει τις εντολές αναζήτησης, ώστε να δίνει απαντήσεις όσο το δυνατόν πιο γρήγορα. Οι πιο δαπανηρές λειτουργίες σε ένα ερώτημα είναι συνήθως οι συνενώσεις (joins). Στην παρούσα άσκηση, υλοποιήσαμε και μελετήσαμε εξειδικευμένους αλγόριθμους συνένωσης (Semijoins και Anti-semijoins) χρησιμοποιώντας τόσο κατακερματισμό (Hashing) όσο και ταξινόμηση (Sort-Merge). Επιπλέον, είδαμε έξυπνους τρόπους για να τρέχει η αναζήτηση πιο γρήγορα: όπως το να φιλτράρουμε τα δεδομένα από την αρχή (για να μην κουβαλάμε περιττές πληροφορίες) και να μεταφέρουμε τα αποτελέσματα απευθείας από το ένα στάδιο στο άλλο, χωρίς να χάνουμε χρόνο αποθηκεύοντάς τα προσωρινά (pipelining).

Θεωρία Τελεστών

Semijoin ($R \ltimes S$): Επιστρέφει τις πλειάδες της σχέσης R για τις οποίες υπάρχει τουλάχιστον μία πλειάδα στη σχέση S με κοινό κλειδί συνένωσης. Σε αντίθεση με το εσωτερικό join, δεν επιστρέφει πεδία από την S και δεν παράγει διπλότυπες εγγραφές της R αν βρεθούν πολλαπλές ταυτίσεις.

Anti-semijoin ($R \rhd S$): Επιστρέφει τις πλειάδες της σχέσης R που δεν συμμετέχουν στη συνένωση, δηλαδή δεν έχουν κανένα ταίρι στη σχέση S . Ισχύει ότι: $R \rhd S = R - (R \ltimes S)$.

Προγραμματιστική Υλοποίηση και Αλγοριθμική Λογική

Η άσκηση αναπτύχθηκε σε γλώσσα Python, κατανεμημένη σε τρία αρχεία (`askisi2_1.py`, `askisi2_2.py`, `askisi2_3.py`), καθένα από τα οποία προσεγγίζει ένα διαφορετικό στάδιο της θεωρίας των συνενώσεων.

Αλγόριθμοι Hash και Sort-Merge (Μέρος 2.1)

Για την υλοποίηση των τελεστών αναπτύχθηκαν δύο διαφορετικές προσεγγίσεις. Η προσέγγιση Hash βασίστηκε στη δομή συνόλου (`set()`) της Python, η οποία προσφέρει πολυπλοκότητα αναζήτησης $O(1)$. Ο αλγόριθμος εισάγει τα κλειδιά της S στο Hash Set και έπειτα ελέγχει αν τα κλειδιά της R υπάρχουν σε αυτό.

Η προσέγγιση Sort-Merge είναι πιο περίπλοκη, καθώς σαρώνει ταυτόχρονα δύο ήδη ταξινομημένες λίστες χρησιμοποιώντας δύο δείκτες (i και j).

```
def sort_merge_semijoin(r, s):
    sorted_r = sorted(r)
    sorted_s = sorted(s)
    i = 0
    j = 0
    result = []

    while i < len(sorted_r) and j < len(sorted_s):
        if sorted_r[i][0] == sorted_s[j][0]:
            result.append(sorted_r[i])
            i += 1
        elif sorted_r[i][0] < sorted_s[j][0]:
            i += 1
        else:
            j += 1

    return result
```

Το πιο σημαντικό σημείο αυτού του κώδικα είναι η στιγμή που συγκρίνουμε τα δεδομένα από τις δύο λίστες για να δούμε αν ταιριάζουν, δηλαδή αν έχουν την ίδια τιμή (if sorted_r[i][0] == sorted_s[j][0]). Σε ένα κανονικό join, θα έπρεπε να χρησιμοποιήσουμε προσωρινούς δείκτες για να διαχειριστούμε τα πολλαπλά ταυτίσματα. Στο Semijoin όμως, μόλις βρεθεί μια ταύτιση, αρκεί να προσθέσουμε την εγγραφή της R στο αποτέλεσμα και να προχωρήσουμε μόνο τον δείκτη i. Δεν απαιτείται backtracking του j, γεγονός που καθιστά τον αλγόριθμο εξαιρετικά αποδοτικό ($O(N+M)$ μετά την ταξινόμηση).

Προηγούμενα Επιλογή και Πραγματικά Δεδομένα (Μέρος 2.2)

Στο δεύτερο μέρος, εφαρμόσαμε τους αλγόριθμους σε πραγματικά δεδομένα (airports.dat και routes.dat). Η αρχιτεκτονική του ερωτήματος απαιτούσε την εύρεση αεροδρομίων που εξυπηρετούν έναν συγκεκριμένο τύπο αεροσκάφους (π.χ. 737).

Σύμφωνα με τους κανόνες βελτιστοποίησης, εκτελέσαμε προηγούμενα επιλογή (Selection Push-down). Αντί να συνενώσουμε πρώτα όλες τις πτήσεις με τα αεροδρόμια, φιλτράραμε το αρχείο routes.dat κρατώντας μόνο τις πτήσεις του ζητούμενου αεροσκάφους, μειώνοντας δραματικά τον όγκο των δεδομένων.

Επιπλέον, αξιοποιήσαμε το γεγονός ότι το airports.dat ήταν ήδη ταξινομημένο, γλιτώνοντας ένα δαπανηρό βήμα ταξινόμησης.

```

filtered_routes = [
    route for route in routes
    if aircraft_type in route[-1].split() and route[5].isdigit()
]
filtered_routes.sort(key=lambda x: int(x[5]))

```

Βασικό Απόσπασμα Κώδικα: Φιλτράρισμα και Ασφάλεια Δεδομένων

Διοχέτευση (Pipelining) και 3-Way Sort-Merge (Μέρος 2.3)

Το πιο προχωρημένο στάδιο της άσκησης απαιτούσε τη συνένωση τριών σχέσεων ($R \bowtie S \bowtie T$) αποφεύγοντας την υλοποίηση (materialization), δηλαδή την εγγραφή του ενδιάμεσου αποτελέσματος ($R \bowtie S$) στη μνήμη ή τον δίσκο.

A) Υλοποίηση Pipelining μέσω Generators

Για να επιτευχθεί η διοχέτευση στην Python, μετατρέψαμε τη συνάρτηση join_r_s σε έναν Generator χρησιμοποιώντας την εντολή yield.

```

def join_r_s_generator(r, s):
    i, j = 0, 0

    while i < len(r) and j < len(s):
        if r[i][0] == s[j][0]:
            temp_j = j
            while temp_j < len(s) and s[temp_j][0] == r[i][0]:
                yield (r[i][0], r[i][1], s[temp_j][1])
                temp_j += 1
            i += 1
        elif r[i][0] < s[j][0]:
            i += 1
        else:
            j += 1

```

Όταν ο αλγόριθμος βρίσκει μια ταύτιση, χρησιμοποιεί έναν προσωρινό δείκτη temp_j για να σαρώσει πιθανά διπλότυπα και δημιουργεί το Καρτεσιανό Γινόμενο των ταυτίσεων (εδώ έχουμε κανονικό join, όχι semijoin). Η εντολή yield "παγώνει" την εκτέλεση της συνάρτησης και περνάει την παραγόμενη πλειάδα απευθείας στην επόμενη φάση του query plan (pipelined_merge_join), γλιτώνοντας έτσι πολύτιμη μνήμη.

B) Ταυτόχρονος 3-Way Sort-Merge

Ως εναλλακτική, υλοποιήθηκε ένας αλγόριθμος που συγχρονίζει τρεις δείκτες (i, j, k) ταυτόχρονα. Η λογική του βασίζεται στην εύρεση του μέγιστου κλειδιού ($\max_key$) τη δεδομένη χρονική στιγμή. Όποια σχέση έχει κλειδί μικρότερο από το $\max_key$ αναγκάζεται να προχωρήσει τον δείκτη της. Μόνο όταν και τα τρία κλειδιά ταυτίζονται, προχωράμε στον συνδυασμό των πλειάδων.

Πειραματική Αξιολόγηση

Τα προγράμματα ελέγχθηκαν εξονυχιστικά χρησιμοποιώντας τόσο τα υποθετικά σύνολα (mock data) που μας δόθηκαν όσο και τα πραγματικά δεδομένα.

Μέρος 2.1 (Ελεγχος Τελεστών): Οι τέσσερις συναρτήσεις ελέγχθηκαν με τις σχέσεις $r = [(1,2),(1,4),(2,5)]$ και $s = [(1,'a'),(1,'c'),(3,'a')]$. Τα αποτελέσματα ήταν ακριβώς τα αναμενόμενα: $\{(1,2),(1,4)\}$ για το `semijoin` και $\{(2,5)\}$ για το `anti-semijoin`.

Μέρος 2.2 (Πραγματικά Δεδομένα): Εκτελώντας το πρόγραμμα με το όρισμα 737, το `sort-merge semijoin` επέστρεψε επιτυχώς 517 εγγραφές αεροδρομίων (ολόκληρες τις πλειάδες), επαληθεύοντας την απαίτηση της εκφώνησης. Η προστασία με την `.isdigit()` απέτρεψε σφάλματα (`ValueErrors`) σε γραμμές του `routes.dat` όπου ο προορισμός ήταν της μορφής `\N`.

Μέρος 2.3 (3-Way Joins): Δοκιμάζοντας και τις δύο υλοποιήσεις (`Pipelining` και `Ταυτόχρονος 3-way`) στο ίδιο σετ δεδομένων, και οι δύο αλγόριθμοι παρήγαγαν ακριβώς το ίδιο τελικό αποτέλεσμα: $\{(1,'b1','c1','d1'), (1,'b1','c2','d1'), (2,'b2','c3','d2'), (3,'b3','c4','d3')\}$.

Συμπεράσματα

Από την υλοποίηση της Άσκησης 2, εξάγονται σημαντικά συμπεράσματα για την αρχιτεκτονική των βάσεων δεδομένων:

Επιλογή Αλγορίθμου: Ο Hash-based αλγόριθμος είναι εξαιρετικά γρήγορος (γραμμικός χρόνος), αλλά εξαρτάται από τη διαθέσιμη μνήμη RAM (για τη δημιουργία του Hash Set). Ο Sort-Merge είναι ιδανικός όταν τα δεδομένα είναι ήδη ταξινομημένα (π.χ. λόγω κάποιου πρωτεύοντος κλειδιού ή B-tree index), καθώς εξοικονομείται το κόστος της ταξινόμησης.

Σημασία του Push-down: Η εφαρμογή του φίλτρου επιλογής πριν από τη συνένωση αποδείχθηκε κρίσιμη στα πραγματικά δεδομένα, καθώς μείωσε το μέγεθος των λιστών, εξοικονομώντας υπολογιστικούς πόρους.

Pipelining vs Materialization: Η υλοποίηση του Pipelining απέδειξε στην πράξη τη θεωρία: η ροή των δεδομένων (stream) από τον έναν τελεστή στον άλλον αποτρέπει τη συμφόρηση στο σύστημα I/O (αποθήκευση στον δίσκο) και επιτρέπει την ταχύτερη επιστροφή των πρώτων αποτελεσμάτων στον χρήστη.

Οδηγίες Εκτέλεσης (How to Run)

Τα προγράμματα έχουν γραφτεί σε standard Python 3. Δεν απαιτείται εγκατάσταση εξωτερικών βιβλιοθηκών. Για την εκτέλεση, ανοίξτε ένα τερματικό στον φάκελο όπου βρίσκονται τα αρχεία .py και τα αρχεία δεδομένων .dat.

Μέρος 2.1 (Ελεγχος Hash και Sort-Merge): Εκτελέστε την παρακάτω εντολή. Θα τυπωθούν τα αποτελέσματα των ελέγχων (tests) για τις 4 συναρτήσεις.

python askisi2_1.py

Μέρος 2.2 (Ερωτήματα στα Αεροδρόμια): Το πρόγραμμα δέχεται ως command-line argument τον τύπο του αεροσκάφους. Παράδειγμα εκτέλεσης για το αεροσκάφος 737:

python askisi2_2.py 737

Το πρόγραμμα θα τυπώσει τον συνολικό αριθμό των αεροδρομίων (αναμένονται 517) και ενδεικτικά τις πρώτες 10 πλήρεις πλειάδες.

Μέρος 2.3 (Δοκιμή Pipelining & 3-Way Join): Εκτελέστε την εντολή για να ελέγξετε την παραγωγή των αποτελεσμάτων του σύνθετου ερωτήματος και να επιβεβαιώσετε ότι και οι δύο προσεγγίσεις βγάζουν ταυτόσημα αποτελέσματα.

python askisi2_3.py